

Northstar Study

Student Performance Prediction Dashboard

Detailed project report

Prepared by Manus AI
Published website: studentdash-vagdh6fx.manus.space
September 2026

Contents

1	Executive summary	1
2	Project context and goals	2
2.1	Primary objectives	2
2.2	Non-goals	2
2.3	Success criteria	2
3	System architecture and technology	3
3.1	Component structure	3
3.2	Runtime state	3
4	Interface anatomy	4
4.1	Snapshot cards	4
4.2	Predictor and reading panel	4
4.3	Pathways and coach note	5
5	Prediction inputs and data contract	5
5.1	Why these features	5
5.2	Data quality considerations	5
6	The single prediction algorithm	6
6.1	Algorithm steps	6
7	Worked example and model flow	6
7.1	Why the result is actionable	7
8	BFS and DFS in the support graph	8
8.1	Breadth-first search	8
8.2	Depth-first search	8
9	User experience and visual system	9
9.1	Typography and hierarchy	9
9.2	Responsive behavior	9
9.3	Accessibility choices	9
10	Implementation excerpts	9
10.1	Maintainability assessment	10
11	Testing and validation	11
11.1	Functional verification notes	11
11.2	Visual verification notes	11
12	Limitations, ethics, and future work	11
12.1	Practical next steps	11
12.2	Recommended product roadmap	12
13	Conclusion and references	12
13.1	References	12

1 Executive summary

Northstar Study is a frontend-only student performance dashboard designed around one simple promise: make a learner's current pattern understandable enough to support a better conversation. The project combines a compact learner snapshot, one transparent weighted-score predictor, and an interactive pathway explorer that demonstrates breadth-first search (BFS) and depth-first search (DFS).

The implementation deliberately avoids a complex machine-learning stack. It uses a single weighted-score calculation that combines attendance, assignment completion, normalized study rhythm, and current average. The output is a projected percentage, a confidence-style indicator, and one of three plain-language bands: **Strong outlook**, **Needs attention**, or **Priority support**. The result is a directional aid rather than a diagnosis, ranking, or admissions decision.

The website is organized like a calm analytics workbook. A navy navigation rail anchors the interface, while a warm ivory canvas contains the hero statement, snapshot cards, predictor controls, factor bars, pathway graph, coach note, and trend line. Northstar cobalt is used as the primary signal color; mint and coral are reserved for semantic meaning.

Project property	Implemented choice
Scope	Frontend-only dashboard with no backend or external API
Prediction	One interpretable weighted-score algorithm
Graph learning	Interactive BFS and DFS traversal of one fixed support graph
Published website	studentdash-vagdh6fx.manus.space

The report documents the product intent, interface architecture, prediction logic, feature roles, BFS/DFS behavior, design system, validation results, limitations, and practical next steps.

2 Project context and goals

Student performance data is often distributed across attendance records, assignment systems, informal teacher notes, and grade books. A teacher may have enough information to form a judgment but not enough time to assemble it into a clear, repeatable workflow. Northstar Study addresses that gap with a small interface that keeps the learner visible while making the model assumptions explicit.

The visual reference for the project was a minimalist student performance analysis dashboard shared through Pinterest and Behance. The reference emphasized clarity, accessibility, functionality, and purposeful placement of dashboard elements [1]. Northstar Study interprets that direction through a custom “Quiet Signal” system: warm paper-like surfaces, a dark navigation spine, compact cards, and a cobalt signal line that connects actions, metrics, and graph traversal.

2.1 Primary objectives

The project has five practical objectives. First, it should provide a readable single-learner snapshot. Second, it should demonstrate a single prediction algorithm rather than hiding complexity behind a model label. Third, it should expose the four inputs that influence the projected result. Fourth, it should make BFS and DFS understandable through an education-support metaphor. Fifth, it should remain small enough to run as a static frontend and easy enough for a student developer to explain.

2.2 Non-goals

The project does not claim to predict a learner’s future with scientific certainty. It does not include a production database, authentication, roster management, automated intervention recommendations, or model training pipeline. It also does not represent a validated institutional assessment instrument. These boundaries are important because the interface is intentionally a teaching-oriented prototype rather than a high-stakes decision system.

2.3 Success criteria

The success criteria are therefore interactional and explanatory. A first-time user should be able to identify the learner, adjust the four inputs, run a prediction, understand the score band, switch between BFS and DFS, and follow the active node in the pathway graph. The codebase should compile cleanly, remain responsive, and avoid unnecessary backend dependencies.

3 System architecture and technology

Northstar Study is implemented as a client-side React application inside a static WebDev project. The architecture favors a single page and a small number of local functions rather than a multi-route application. The page owns the learner state, prediction state, traversal state, and mobile navigation state.

Layer	Technology	Responsibility
Presentation	React 19 + JSX	Renders the dashboard, cards, controls, graph nodes, and responsive navigation
Styling	Tailwind import + authored CSS	Owns the Quiet Signal tokens, layout, responsive breakpoints, and states
Icons	Lucide React	Provides compact interface cues without custom image dependencies
Routing	Single / route	Keeps the MVP intentionally simple
Build	Vite + TypeScript	Type checking, development server, and production bundle
Deployment	Manus WebDev static hosting	Serves the published frontend at the project domain

3.1 Component structure

The primary page component contains a small reusable StatCard component and local helper functions. `getPrediction` is responsible for the weighted-score algorithm. The fixed `nodeOrder` array provides the support graph's node labels and semantic tones. Two sequence arrays, `bfsSequence` and `dfsSequence`, define the visible traversal order. This structure keeps the model and graph logic easy to find for a learner reading the code.

```
const weightedScore =
  attendance * 0.3 +
  assignments * 0.25 +
  Math.min(studyHours * 5, 100) * 0.15 +
  average * 0.3;

const projected = Math.round(weightedScore);
```

The website uses no server routes, database calls, or runtime prediction services. All values are computed in the browser from the current slider state. This makes the demo deterministic and fast, but it also means that values are not persisted between sessions.

3.2 Runtime state

The UI maintains four input values, one prediction object, one active traversal tab, one traversal step index, and one mobile navigation boolean. When the user changes a slider, the factor display updates immediately. When the user selects “Run prediction,” the result panel is recalculated. When the user selects BFS or DFS, the sequence and active node reset to the first step.

4 Interface anatomy

The interface follows a persistent rail plus analysis canvas pattern. The rail contains the Northstar Study identity, workspace navigation, quick learner view, coach tip, settings, and teacher account. The main canvas begins with a breadcrumb and date context, then moves into a large hero statement and four learner snapshot cards.

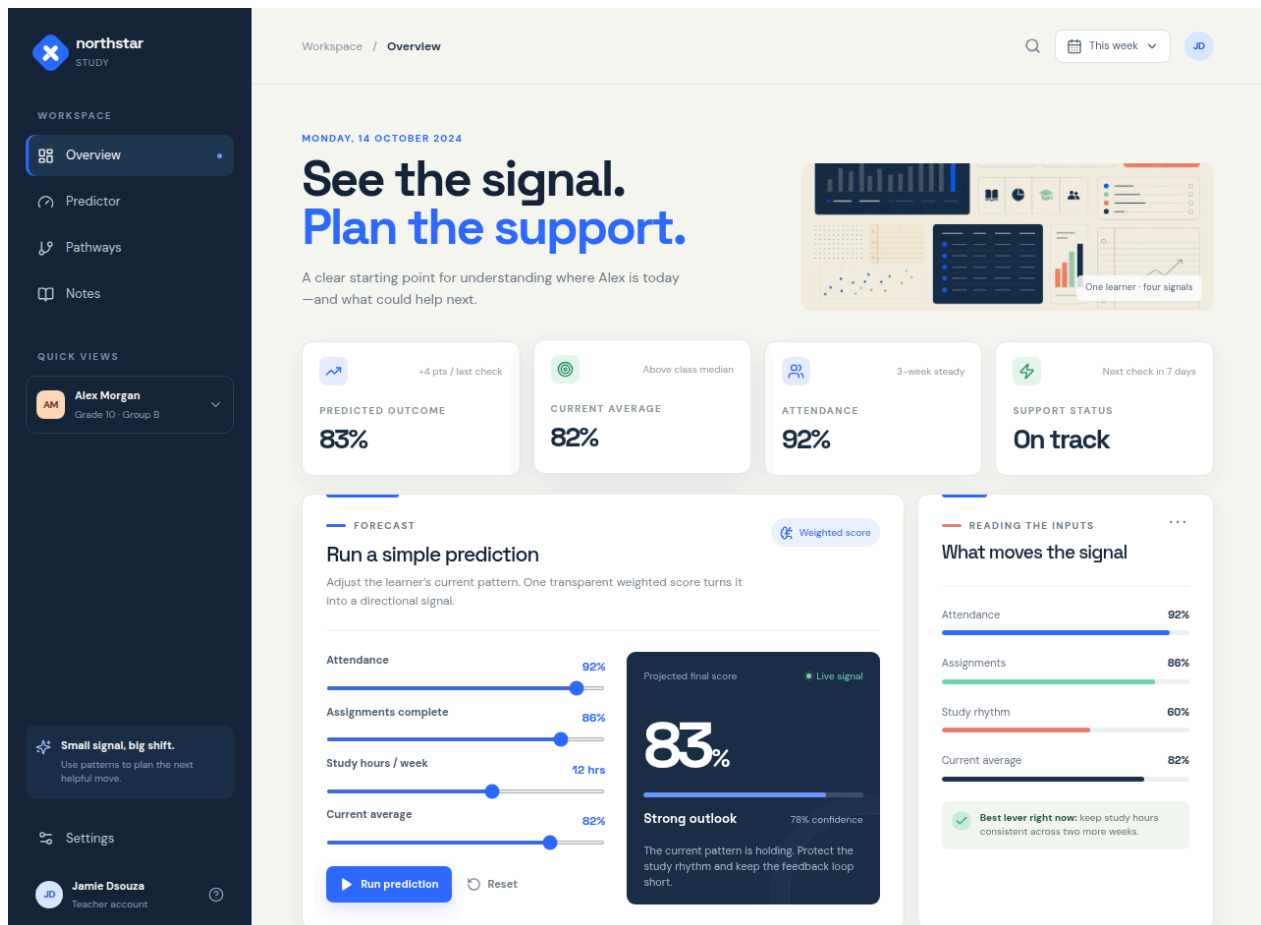


图 1 Full dashboard view: the navy rail, warm ivory canvas, metric snapshot, predictor, pathway graph, and supporting cards.

The hero copy, “See the signal. Plan the support.”, is not a generic welcome message. It communicates the product’s two-step mental model: first understand the current signal, then choose a support action. The interface keeps the learner name and group close to the navigation so that the surrounding analytics content remains grounded in a person rather than an abstract dataset.

4.1 Snapshot cards

The four top-level cards create a fast scan path. “Predicted outcome” is the primary output. “Current average” anchors the score in present academic performance. “Attendance” expresses consistency. “Support status” translates the score band into an operational state. Each card uses a small icon tile, a quiet secondary detail line, an uppercase label, and a large metric numeral.

4.2 Predictor and reading panel

The predictor occupies the widest content region because it is the central task. To its right, the “What moves the signal” panel repeats the same factors as horizontal bars. This duplication is intentional: the left card allows adjustment, while the right card makes the relative reading easy to scan.

4.3 Pathways and coach note

The pathway section turns the algorithm requirement into a visible education metaphor. The bottom cards then re-humanize the dashboard: the coach note reminds educators that prediction should start a conversation, while the trend panel frames the current signal as a trajectory rather than a final verdict.

5 Prediction inputs and data contract

The predictor consumes four numeric inputs. All four values are constrained to simple ranges in the UI. Attendance, assignment completion, and current average use a 0–100 scale. Study hours use a 0–20 weekly range and are converted to a 0–100 rhythm score by multiplying by five and capping at 100.

Input	UI range	Model scale	Interpretation
Attendance	0–100%	0–100	Daily consistency
Assignments complete	0–100%	0–100	Follow-through on required work
Study hours / week	0–20 hours	hours × 5, capped at 100	Repeatable effort outside class
Current average	0–100%	0–100	Current academic anchor

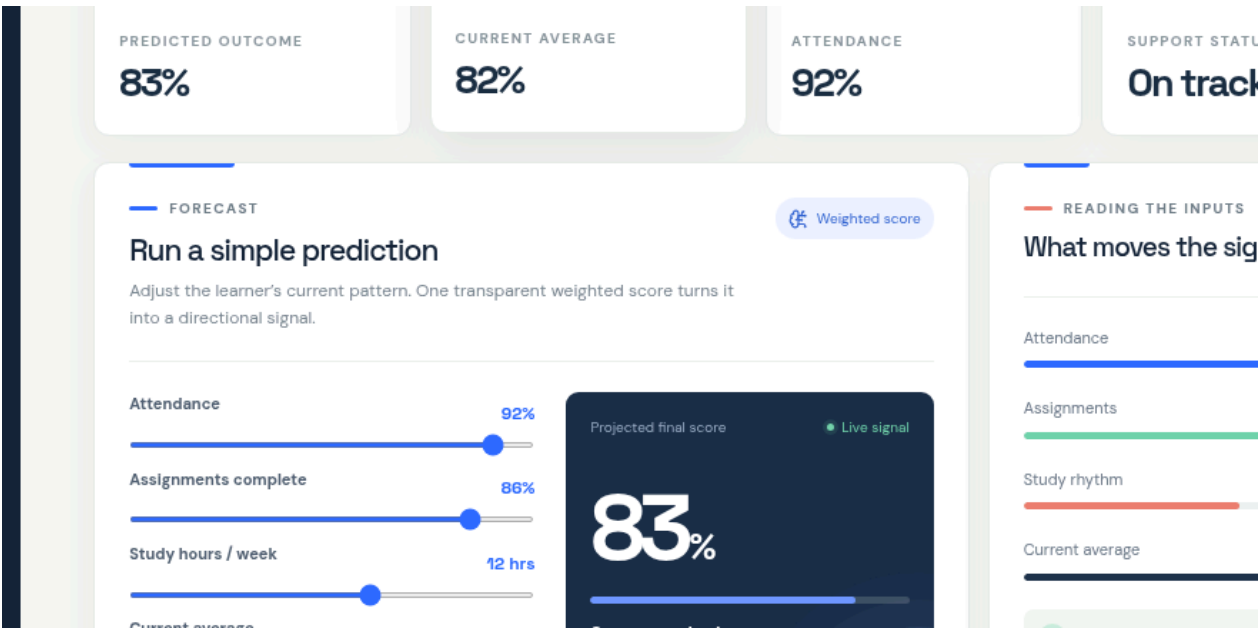


图 2 Predictor snippet: four visible controls and the dark result panel used by the live website.

5.1 Why these features

Attendance and assignments represent observable participation and completion behaviors. Study rhythm adds a controllable behavior that is not reducible to a single grade. Current average provides a direct academic anchor. The feature set is intentionally small enough to explain on one screen and to discuss with a learner.

5.2 Data quality considerations

The current frontend assumes the values are already meaningful and correctly entered. It does not handle missing values, contradictory records, late updates, or differences between subjects. A production extension would need explicit validation rules, source timestamps, data provenance, and a strategy for missing or stale observations.

6 The single prediction algorithm

The website uses one deterministic weighted-score algorithm. This is the only prediction algorithm in the project. The model does not train on a dataset and does not update its coefficients at runtime. Its value is interpretability: every output can be traced to four visible values and four fixed weights.

$$P = (A \times 0.30) + (C \times 0.25) + (S \times 0.15) + (G \times 0.30)$$

In the equation, P is the projected score, A is attendance, C is assignment completion, S is the normalized study rhythm, and G is the current average. The weights sum to 1.00, so the resulting score remains on the same 0–100 scale as the inputs.

6.1 Algorithm steps

1. Read the four input values from React state.
2. Convert weekly study hours into a comparable percentage by multiplying by five and capping the result at 100.
3. Multiply each feature by its fixed weight.
4. Add the weighted terms.
5. Round the result to the nearest whole number.
6. Map the rounded score to one of three bands.
7. Generate concise interpretive copy for the result panel.

```
const band = projected >= 80
  ? "Strong outlook"
  : projected >= 65
    ? "Needs attention"
    : "Priority support";
```

The model is deliberately not described as “accurate” because no training dataset or evaluation benchmark is part of this MVP. The correct claim is that it is transparent, deterministic, and useful for demonstration.

7 Worked example and model flow

The default website state uses attendance of 92%, assignment completion of 86%, 12 study hours per week, and a current average of 82%. Study rhythm becomes 60% because $12 \times 5 = 60$. The weighted terms are 27.6, 21.5, 9.0, and 24.6. Their sum is 82.7, which rounds to 83%.

Feature	Value	Weight	Contribution
Attendance	92	0.30	27.6
Assignments	86	0.25	21.5
Study rhythm	60	0.15	9.0
Current average	82	0.30	24.6
Projected score	82.7	1.00	83 after rounding

The website displays 83%, a “Strong outlook” band, and a 78% confidence-style indicator. That indicator is not a statistically calibrated probability; it is an interface cue derived from the projected score. It should be treated as a visual confidence signal, not a validated estimate of predictive accuracy.

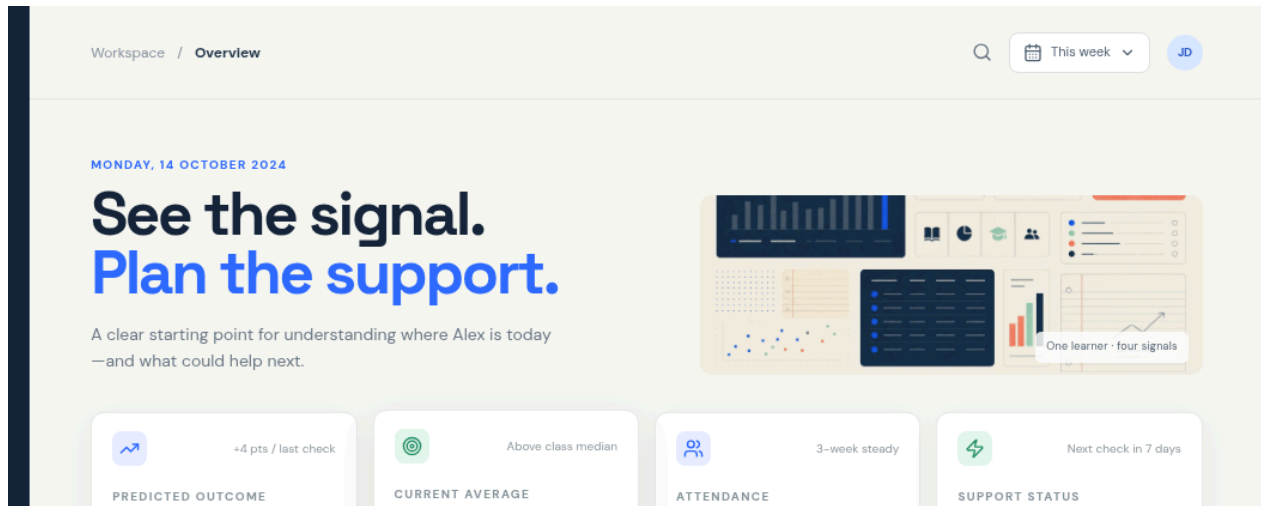


图 3 Hero and snapshot snip: the interface frames the score as one signal among several learner indicators.

7.1 Why the result is actionable

The recommendation is intentionally modest: protect study rhythm and keep the feedback loop short. A high score does not trigger an automated intervention. Instead, it suggests that the current pattern is holding and that consistency may be the most useful next conversation.

8 BFS and DFS in the support graph

The graph section is not another prediction algorithm. BFS and DFS are used as visible traversal demonstrations over a fixed graph of possible support actions. The graph begins at Start and connects to nodes such as Attendance, Study plan, Tutoring, Practice set, and Ready check.

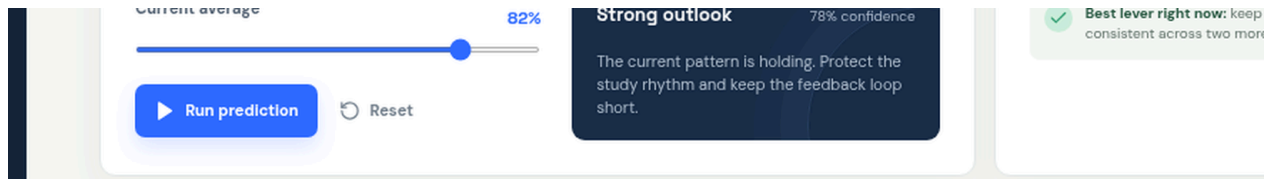


图 4 Pathways snip: the website presents traversal as a learner-support graph with an active node and step panel.

8.1 Breadth-first search

BFS explores level by level. In the project's sequence, it visits Start, then Attendance and Study plan, before moving to Tutoring, Practice set, and Ready check. In a real graph implementation, a queue would store the next nodes to visit.

```
queue = [Start]
while queue is not empty:
    current = queue.pop_front()
    visit(current)
    add unvisited neighbors to queue
```

The interface uses a fixed BFS sequence array for clarity and animation-free demonstration. The visible effect is that nearby support options are considered before the path goes deeper.

8.2 Depth-first search

DFS follows one branch as deeply as possible before backtracking. The project's visible DFS sequence is Start, Attendance, Tutoring, Practice set, Ready check, and Study plan. A stack or recursive call structure is the conventional implementation pattern.

```
stack = [Start]
while stack is not empty:
    current = stack.pop()
    visit(current)
    push unvisited neighbors onto stack
```

The website's value is pedagogical: the same support graph can be read with two different traversal lenses, making algorithm behavior easier to relate to the idea of exploring intervention options.

9 User experience and visual system

The interface uses the Quiet Signal design direction defined for the project. The surface is warm ivory rather than stark white. The navigation rail is ink navy, providing orientation and contrast. Northstar cobalt is the primary signal color and appears in the active nav state, section markers, primary actions, score emphasis, sliders, and active graph nodes. Mint and coral are semantic only. Mint communicates healthy or protective progress. Coral communicates attention or risk. This semantic discipline keeps the palette from becoming decorative noise.

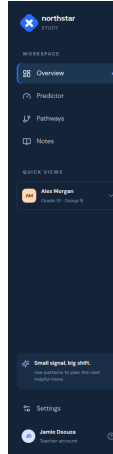


图 5 Sidebar snippet: the persistent rail establishes workspace hierarchy and the Northstar Study identity.

9.1 Typography and hierarchy

Space Grotesk is used for display headings, metrics, and compact labels. DM Sans is used for body copy and explanatory text. The contrast between the geometric display face and the readable sans-serif body creates the editorial dashboard tone seen throughout the website.

9.2 Responsive behavior

On smaller screens, the persistent sidebar becomes a slide-in navigation panel with a scrim and menu button. The four metric cards become a two-column grid. The predictor card stacks its controls above the result panel, while the graph canvas remains horizontally scrollable so the node layout does not collapse into an unreadable vertical list.

9.3 Accessibility choices

The implementation includes visible focus styles, labeled controls, button labels, alternative text for meaningful images, and prefers-reduced-motion handling. The current MVP can be strengthened further with more explicit form descriptions, keyboard testing for the graph controls, and announcements for traversal step changes.

10 Implementation excerpts

The project keeps the codebase intentionally small. The following excerpt shows the central scoring function, which is readable without importing a model framework.

```
function getPrediction(inputs: Inputs) {  
  const weightedScore =  
    inputs.attendance * 0.3 +  
    inputs.assignments * 0.25 +  
    Math.min(inputs.studyHours * 5, 100) * 0.15 +  
    inputs.average * 0.3;  
}
```

```
const projected = Math.round(weightedScore);
const band = projected >= 80
  ? "Strong outlook"
  : projected >= 65
    ? "Needs attention"
    : "Priority support";

return { projected, band };
}
```

The graph state is similarly direct. The active tab selects one of two sequences, and the current step index selects the active node. This is enough to demonstrate the algorithms without adding a graph library or a backend service.

```
const sequence = activeTab === "bfs"
  ? bfsSequence
  : dfsSequence;

const activeNodeId =
  sequence[Math.min(stepIndex, sequence.length - 1)];
```

The frontend also exposes a reset action. Resetting the demo restores the original learner inputs and recalculates the default 83% signal. This supports classroom demonstrations because the interface can return to a known state quickly.

10.1 Maintainability assessment

The current structure is appropriate for a teaching prototype: the model is easy to locate, constants are visible, and there are no hidden services. If the project grows, the next refactor should separate the prediction utility, traversal utility, and dashboard presentation into small modules while preserving the same vocabulary.

11 Testing and validation

Validation was performed at three levels. First, the TypeScript compiler was run through `pnpm check`. Second, the production build was generated with `pnpm build`. Third, the live preview was inspected on desktop and mobile viewport sizes.

Check	Evidence	Result
TypeScript check	<code>tsc --noEmit</code> completed without errors	Pass
Production build	Vite bundle and server bundle completed	Pass
Desktop layout	Dashboard rendered across the full overview	Pass
Mobile layout	Cards stacked, nav collapsed, graph remained usable	Pass
Prediction interaction	Sliders, Run prediction, and Reset are wired	Pass in implementation review
Traversal interaction	BFS/DFS tabs and Next step update visible state	Pass in implementation review

The production build emitted a chunk-size warning because the generated JavaScript bundle exceeded the default advisory threshold. This is not a build failure, but it is a reasonable future optimization target through code splitting or dependency review.

11.1 Functional verification notes

The default state produces 83% from the visible inputs. Lowering attendance or current average moves the score and can move it into another band. Switching from BFS to DFS resets the traversal step and changes the active path order. The Reset button returns the predictor to its initial state.

11.2 Visual verification notes

The desktop layout preserves the navy rail and large analysis canvas. The mobile layout hides the rail behind a menu action, reduces the hero scale, stacks the predictor and factor panels, and preserves the pathway graph through horizontal overflow rather than compressing node labels.

12 Limitations, ethics, and future work

Northstar Study is intentionally limited. The score weights are authored for interpretability, not estimated from evidence in this prototype. The confidence-style indicator is not calibrated. Study hours are a rough self-report. The support graph is a fixed teaching example rather than a domain-complete intervention model.

These limitations matter because student performance data can influence how educators talk about learners. A projection should never become a label, and a low score should not be used as a reason to reduce opportunity. The project's own coach note expresses the correct stance: "Predictions are prompts for a conversation, not labels."

12.1 Practical next steps

A first extension would add a CSV roster import with clear validation and preview before any prediction is run. A second would add persistence for prediction history and teacher notes, with privacy and access controls. A third would support printable learner summaries. A fourth would replace the authored weights with a validated model only after a representative dataset, documented evaluation protocol, fairness review, and stakeholder oversight are available.

12.2 Recommended product roadmap

Phase	Capability	Reason
1	Roster import and validation	Move from one learner demo to repeatable classroom use
2	Saved notes and prediction history	Support longitudinal teacher workflows
3	Accessibility and keyboard audit	Improve inclusive use and compliance confidence
4	Evidence-based model evaluation	Test calibration, performance, and fairness before higher-stakes use

13 Conclusion and references

Northstar Study demonstrates how a small frontend can combine product design, a transparent prediction method, and classic graph algorithms into one coherent learning tool. Its strength is not model complexity. Its strength is that the user can see the inputs, understand the calculation, compare BFS and DFS behavior, and leave with a practical next step. The published website is available at <https://studentdash-vagdh6fx.manus.space/>. The accompanying QR code generated for the project points to the same URL.

13.1 References

1. Ciphernutz IT Services. “Student Performance Analysis Dashboard.” Behance.
2. Pinterest reference pin supplied for the project.
3. Northstar Study published website.
4. MDN Web Docs. JavaScript reference.
5. React documentation.

Northstar Study · A small signal for better support.